Capstone Project 1: E-Commerce Platform Modernization

Problem Statement

A monolithic e-commerce application is struggling with:

- Slow deployments and downtime during releases
- Poor scalability during peak traffic (sales/events)
- No proper CI/CD automation
- Limited observability and debugging capability

Goal:
Modernize the application using **microservices architecture**, containerization, Kubernetes (GKE/AKS), and CI/CD pipelines.

Scenarios

- Black Friday traffic spike
- Failed deployment causing downtime
- Service latency issues (checkout/payment)
- Security vulnerabilities in container images
- Need for blue-green or canary deployments

Case Studies

Case 1: Monolith to Microservices Migration

Problem: Tight coupling, hard to scale
Solution Steps:

- Identify modules: user, product, cart, payment
- Split into independent services
- Containerize each service using Docker
- Deploy independently in Kubernetes

Case 2: Containerization

Problem: App runs only in specific environments
Solution:

- Create Dockerfile
- Build images
- Push to container registry
- Ensure stateless services

Case 3: Kubernetes Deployment

Problem: Manual deployments
Solution:

- Create Deployment YAML
- Define replicas, labels
- Use Service for internal/external access
- Apply using kubectl apply -f

Case 4: CI/CD Pipeline Setup

Problem: Manual build & deploy
Solution:

- Use GitHub Actions / Jenkins
- Pipeline stages:
    - Build
    - Test
    - Security scan
    - Docker build
    - Push image
    - Deploy to GKE

Case 5: Auto Scaling

Problem: Performance degradation during traffic spikes
Solution:

- Configure HPA (Horizontal Pod Autoscaler)
- Scale based on CPU/memory usage

Case 6: Load Balancing & Ingress

Problem: External access not controlled
Solution:

- Configure Ingress Controller
- Define routing rules
- Enable SSL/TLS

Case 7: Observability & Monitoring

Problem: No visibility into system
Solution:

- Enable GKE Monitoring / GCP Monitoring
- Monitor CPU, memory, latency
- Enable logging via Cloud Logging

Case 8: Security Hardening

Problem: Vulnerable containers and open access
Solution:

- Use RBAC in Kubernetes
- Implement Network Policies
- Scan images using tools (Trivy, etc.)
- Store secrets securely

Case 9: Fault Tolerance

Problem: Application crashes lead to downtime
Solution:

- Configure liveness & readiness probes
- Enable pod restart policies
- Use replica sets

Case 10: Deployment Strategy

Problem: Risky updates
Solution:

- Explain Rollback on failure & GKE Commands for it

Final Conclusion

- Successfully transformed monolithic system into microservices
- Achieved **automated CI/CD pipeline**
- Improved **scalability, availability, and resilience**
- Reduced deployment time from hours to minutes
- Enhanced **security and monitoring capabilities**

Criteria	Description	Weightage	Excellent (Full Marks)	Average	Needs Improvement
1. Architecture Design (Microservices)	Ability to break monolith into services	10	Clear modular design (user, cart, payment), well-decoupled	Partial decomposition	Still monolithic / unclear
2. Containerization (Docker)	Dockerfile, image creation, portability	10	Optimized Dockerfile, small image, best practices	Basic Dockerfile	Errors / not working
3. Kubernetes Deployment	YAML, deployments, services	10	Proper labels, replicas, services, working deployment	Minor issues	Deployment fails
4. CI/CD Pipeline Implementation	Automation using GitHub Actions / Jenkins / Cloud Build	15	Full pipeline: build → test → scan → deploy automated	Partial automation	Manual or broken pipeline

Criteria	Description	Weightage	Excellent (Full Marks)	Average	Needs Improvement
5. Auto Scaling (HPA)	Handling traffic spikes	8	HPA correctly configured and tested	Configured but not tested	Not implemented
6. Load Balancing & Ingress	External access, routing, TLS	8	Ingress + SSL + routing working	Basic service exposed	No proper routing
7. Observability & Monitoring	Logging & monitoring setup	8	Metrics + logs + dashboards + alerts	Partial visibility	No monitoring
8. Security Implementation	RBAC, image scanning, secrets	10	RBAC + NetworkPolicy + secure secrets + scan	Basic security	Open/unsecured setup
9. Fault Tolerance & Resilience	Probes, restart policies	6	Liveness/readiness + self-healing validated	Partial setup	No resilience
10. Deployment Strategy (Blue-Green/Canary)	Safe deployments & rollback	5	Canary/Blue-Green + rollback tested	Only rolling update	No strategy
11. Problem-Solution Mapping	Alignment with given scenarios	5	All scenarios mapped with clear solutions	Partial mapping	Missing scenarios
12. Documentation & Presentation	Clarity, diagrams, explanation	5	Clear architecture diagram + structured explanation	Basic docs	Poor/no documentation